

Assignment 2 (Weeks 3 and 4)

Solutions

Note on SQL notation. The assignment schema uses a table named `Order`. Since `ORDER` is a reserved keyword in standard SQL, I write it as "`Order`" using standard SQL delimited-identifier syntax.

Part I: Short Answer / Reasoning Questions

Q1

The query returns 0 because `NULL` does not mean an ordinary value. It means that the value is unknown or missing, so the comparison `phone_number = NULL` does not evaluate to true for any row. In SQL, comparisons with `NULL` usually evaluate to `UNKNOWN`, and the `WHERE` clause keeps only rows where the condition is `TRUE`.

The correct query is:

```
SELECT COUNT(*)  
FROM Student  
WHERE phone_number IS NULL;
```

In general, `NULL` is excluded from most aggregate computations because it represents missing information, not a numeric zero or an empty string. For example, `AVG(salary)` should average only known salaries; treating missing salaries as zero would change the meaning of the result. Similarly, ordinary comparison operators cannot say whether an unknown value is equal to, less than, or greater than another value.

Q2

Yes, the student is correct: in this query, `DISTINCT` is redundant.

```
SELECT DISTINCT department, COUNT(*)  
FROM Employee  
GROUP BY department;
```

The `GROUP BY department` clause already produces one output row per department. The `DISTINCT` keyword would be applied after grouping to the final output pairs (`department`, `COUNT(*)`). Since each department appears only once after grouping, there are no duplicate output rows for `DISTINCT` to remove.

Q3

If the `INNER JOIN` and `LEFT JOIN` give different row counts when `Customer` is the left table, the data contains at least one customer with no matching order. The `INNER JOIN` keeps only customers that have a matching row in "Order", while the `LEFT JOIN` keeps every customer and fills the order columns with `NULL` when no match exists.

The `LEFT JOIN` returns more rows specifically when one or more customers have never placed an order. Matching customers still appear once per matching order in both joins, but unmatched customers appear only in the `LEFT JOIN` result.

Q4

The query is invalid in standard SQL because `employee_name` is neither included in the `GROUP BY` clause nor wrapped in an aggregate function.

```
SELECT department, employee_name, AVG(salary)
FROM Employee
GROUP BY department;
```

After grouping by department, SQL has one group for each department, but each group may contain many employees with different names. If SQL allowed the query as written, it would be unclear which `employee_name` should be shown beside the department's average salary.

A corrected version that asks for the average salary per department is:

```
SELECT department, AVG(salary) AS average_salary
FROM Employee
GROUP BY department;
```

Q5

The error is that the query tries to use `AVG(salary)` inside the `WHERE` clause:

```
SELECT department, AVG(salary)
FROM Employee
WHERE AVG(salary) > 80000
GROUP BY department;
```

`WHERE` filters individual rows before groups and aggregate values are created. Since `AVG(salary)` exists only after the rows have been grouped, the aggregate condition must be placed in `HAVING`.

The corrected query is:

```
SELECT department, AVG(salary) AS average_salary
FROM Employee
GROUP BY department
HAVING AVG(salary) > 80000;
```

Q6

(a) A non-correlated subquery is more appropriate for finding products priced above the overall average product price. The average price is a single platform-wide value, so it does not need to depend on each outer product row.

```
SELECT product_id, product_name, price
FROM Product
WHERE price > (
    SELECT AVG(price)
    FROM Product
);
```

(b) A correlated subquery is more appropriate for finding employees who earn more than the average salary of their own department, because the inner average depends on the department of the current outer employee row.

```
SELECT e.employee_id, e.employee_name, e.department, e.salary
FROM Employee e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM Employee e2
    WHERE e2.department = e.department
);
```

The key performance implication is that a correlated subquery may be evaluated logically once for every row of the outer query. When the outer table is large, this can be much more expensive than a non-correlated subquery, although some database optimizers can rewrite correlated queries into more efficient join-like plans.

Part II: Long Answer Question

Schema:

```
Customer(customer_id, name, email, phone)
Restaurant(restaurant_id, name, cuisine_type, rating)
MenuItem(item_id, restaurant_id, item_name, price)
"Order"(order_id, customer_id, restaurant_id, order_date, status)
OrderItem(order_id, item_id, quantity)
```

(a) Customers who have never placed an order

```
SELECT c.customer_id, c.name, c.email, c.phone
FROM Customer c
LEFT JOIN "Order" o
    ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;
```

I used a LEFT JOIN because the query must preserve customers even when they have no matching order. The WHERE o.order_id IS NULL condition then keeps only the customers for whom no order row was found.

(b) Average order value for each restaurant with more than 5 orders

```
WITH order_values AS (
    SELECT
        o.order_id,
```

```

        o.restaurant_id,
        SUM(mi.price * oi.quantity) AS order_value
FROM "Order" o
JOIN OrderItem oi
    ON oi.order_id = o.order_id
JOIN MenuItem mi
    ON mi.item_id = oi.item_id
    AND mi.restaurant_id = o.restaurant_id
GROUP BY o.order_id, o.restaurant_id
)
SELECT
    r.restaurant_id,
    r.name AS restaurant_name,
    AVG(ov.order_value) AS average_order_value
FROM Restaurant r
JOIN order_values ov
    ON ov.restaurant_id = r.restaurant_id
GROUP BY r.restaurant_id, r.name
HAVING COUNT(*) > 5;

```

I first used a common table expression to compute one value per order, because an order value is the sum of `price * quantity` across its items. The `HAVING` clause is needed because the condition “more than 5 orders” filters grouped restaurant results, not individual rows.

(c) Restaurants with average order value above the platform-wide average

```

WITH order_values AS (
    SELECT
        o.order_id,
        o.restaurant_id,
        SUM(mi.price * oi.quantity) AS order_value
    FROM "Order" o
    JOIN OrderItem oi
        ON oi.order_id = o.order_id
    JOIN MenuItem mi
        ON mi.item_id = oi.item_id
        AND mi.restaurant_id = o.restaurant_id
    GROUP BY o.order_id, o.restaurant_id
),
restaurant_aov AS (
    SELECT
        restaurant_id,
        AVG(order_value) AS average_order_value
    FROM order_values
    GROUP BY restaurant_id
    HAVING COUNT(*) > 5
),
platform_aov AS (
    SELECT AVG(order_value) AS platform_average_order_value
    FROM order_values
)
SELECT
    r.restaurant_id,
    r.name AS restaurant_name,
    ra.average_order_value
FROM restaurant_aov ra

```

```

JOIN Restaurant r
  ON r.restaurant_id = ra.restaurant_id
CROSS JOIN platform_aov pa
WHERE ra.average_order_value > pa.platform_average_order_value;

```

This query reuses the per-order totals from part (b), then compares each restaurant's average order value with the single platform-wide average order value. A non-correlated construction is appropriate because the platform average is independent of the current restaurant and can be computed once.

(d) Most ordered menu item by total quantity for each restaurant

```

WITH item_totals AS (
  SELECT
    mi.restaurant_id,
    mi.item_id,
    mi.item_name,
    SUM(oi.quantity) AS total_quantity
  FROM MenuItem mi
  JOIN OrderItem oi
    ON oi.item_id = mi.item_id
  GROUP BY mi.restaurant_id, mi.item_id, mi.item_name
),
ranked_items AS (
  SELECT
    restaurant_id,
    item_id,
    item_name,
    total_quantity,
    RANK() OVER (
      PARTITION BY restaurant_id
      ORDER BY total_quantity DESC
    ) AS quantity_rank
  FROM item_totals
)
SELECT
  r.restaurant_id,
  r.name AS restaurant_name,
  ri.item_id,
  ri.item_name,
  ri.total_quantity
FROM ranked_items ri
JOIN Restaurant r
  ON r.restaurant_id = ri.restaurant_id
WHERE ri.quantity_rank = 1;

```

I used a window function because this is a top-per-group problem: the query must rank menu items separately within each restaurant. A basic `GROUP BY + MAX` approach can find the maximum quantity per restaurant, but it does not by itself return the menu item row associated with that maximum, and it handles ties awkwardly without an additional join or ranking step.

Part III: Normalization

Q1

The functional dependencies implied by the scenario are:

- `student_id -> student_name`
- `course_id -> course_name, instructor_id`
- `instructor_id -> instructor_name, instructor_office`
- `(student_id, course_id) -> grade, enrollment_date`

Using these dependencies together, `(student_id, course_id)` determines all attributes in the original `Enrollment` table, so `(student_id, course_id)` is the natural composite key.

Q2

Yes, the table is in 1NF if each column stores a single atomic value and each row represents one student's enrollment in one course. There are no repeating groups or multi-valued attributes in the given design.

Q3

The partial dependencies that violate 2NF are:

- `student_id -> student_name`
- `course_id -> course_name, instructor_id`

These are partial dependencies because the primary key is the composite key `(student_id, course_id)`, but these non-key attributes depend on only one part of that key. This creates anomalies because the same student or course facts are repeated across many enrollment rows. For example, if a course name changes, every enrollment row for that course must be updated; missing one row would leave inconsistent course names in the table.

Q4

The transitive dependency that violates 3NF is:

`course_id -> instructor_id -> instructor_name, instructor_office`

Equivalently, instructor details depend on `instructor_id`, which is not the key of the original enrollment table. This causes anomalies because the same instructor name and office may be repeated in many rows. For example, if an instructor moves to a new office, every row mentioning that instructor must be updated, or the table may show two different offices for the same instructor.

Q5

A 3NF decomposition is:

- `Student(student_id, student_name)`
- `Instructor(instructor_id, instructor_name, instructor_office)`
- `Course(course_id, course_name, instructor_id)`
- `Enrollment(student_id, course_id, grade, enrollment_date)`

The `Student` table is justified by `student_id -> student_name`. The `Instructor` table is justified by `instructor_id -> instructor_name, instructor_office`. The `Course` table is justified by `course_id -> course_name, instructor_id`. The decomposed `Enrollment` table keeps the fact that `(student_id, course_id) -> grade, enrollment_date`; its composite primary key also connects each enrollment to the corresponding student and course.

Part IV: Design Justification

In Part II(a), I could have expressed customers who never placed an order using either a `LEFT JOIN` with a `NULL` check or a `NOT EXISTS` subquery. I chose the `LEFT JOIN` form because it makes the missing-match idea visible: preserve every customer, then keep only rows where no order joined. It is also readable for this schema because `order_id` is a primary key, so checking `o.order_id IS NULL` cannot accidentally match a real order. A `NOT EXISTS` version would be equally valid and can be preferable when duplicate rows or nullable join columns make an anti-join harder to reason about; many optimizers convert both forms to similar plans.

Normalizing the `Enrollment` table into `Student`, `Course`, `Instructor`, and `Enrollment` removes repeated facts and prevents inconsistent updates, but it makes ordinary reporting less convenient. A simple list of student names, course names, instructor offices, grades, and dates now requires several joins instead of one table scan. Real systems sometimes keep limited denormalization for performance, simpler analytics, or historical snapshots, as long as the duplicated data is controlled by application logic, constraints, or refresh processes.